

Week 9 Tutorial

COMP10001 – Foundations of Computing

Semester 1, 2026

Clement Chau

- "List" Comprehensions
- Iterators and Itertools

Revision: List Comprehensions

```
cashier_3 = []  
for item in cart:  
    if item % 2 == 0:  
        cashier_3.append(item)
```

Non-list comprehension

```
cashier_3 = [item for item in cart if item % 2 == 0]
```

List comprehension

Revision: (Types of) List Comprehensions

sem1-2025 > week-9 >  list_comprehension.py > ...

```
1  # List Comprehension
2  l = [i for i in range(10)]
3
4  # List Comprehension with If Condition
5  l = [i for i in range(10) if i % 2 == 0]
6
7  # List Comprehension with If/Else Condition
8  l = [i if i % 2 == 0 else i * i for i in range(10)]
9
10 # Nested Comprehension
11 l = [(i, j) for i in range(10) for j in range(10)]
```

Revision: More types of List Comprehensions

```
13     # Dictionary Comprehension
14     d = {i: i * i for i in range(10)}
15
16     # Set Comprehension
17     s = {i for i in range(10) if i % 2 == 0}
18
19     # Generator Expression (Not Examinable)
20     g = (i for i in range(10) if i % 2 == 0)
21
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
{0, 2, 4, 6, 8}
<generator object <genexpr> at 0x000002C6921235E0>
```

Revision: Iterators

▶ Run

PYTHON

⌵

```
1 list1 = [1, 5]
2 my_iterator = iter(list1)
3 print(next(my_iterator))
4 print(next(my_iterator))
5
6 # This will generate an error because we have reached the end.
7 print(next(my_iterator))
```

⌵

```
1
5
Traceback (most recent call last):
  File "/home/main.py", line 7, in <module>
    print(next(my_iterator))
          ~~~~^~~~~~
StopIteration
```

✗ Program exited with code 1

Revision: Iterables

- **list**
- **str**
- **tuple**
- **set**
- **dict**
- file objects (We'll talk more about this next week!)

Revision: Iterators vs Sequences

Sequences:

- Have *random access* (you can access any element in the sequence, as many times as you like)
- No position tracking within the sequence
- You can use `len()` to calculate the length
- Must be finite
- You can traverse it many times

Iterators:

- No *random access*
- Remembers where you are up to
- Cannot use `len()`
- Can be infinite
- You can only traverse it once, forwards.

Revision: Itertools (Cycle)

▶ Run

PYTHON



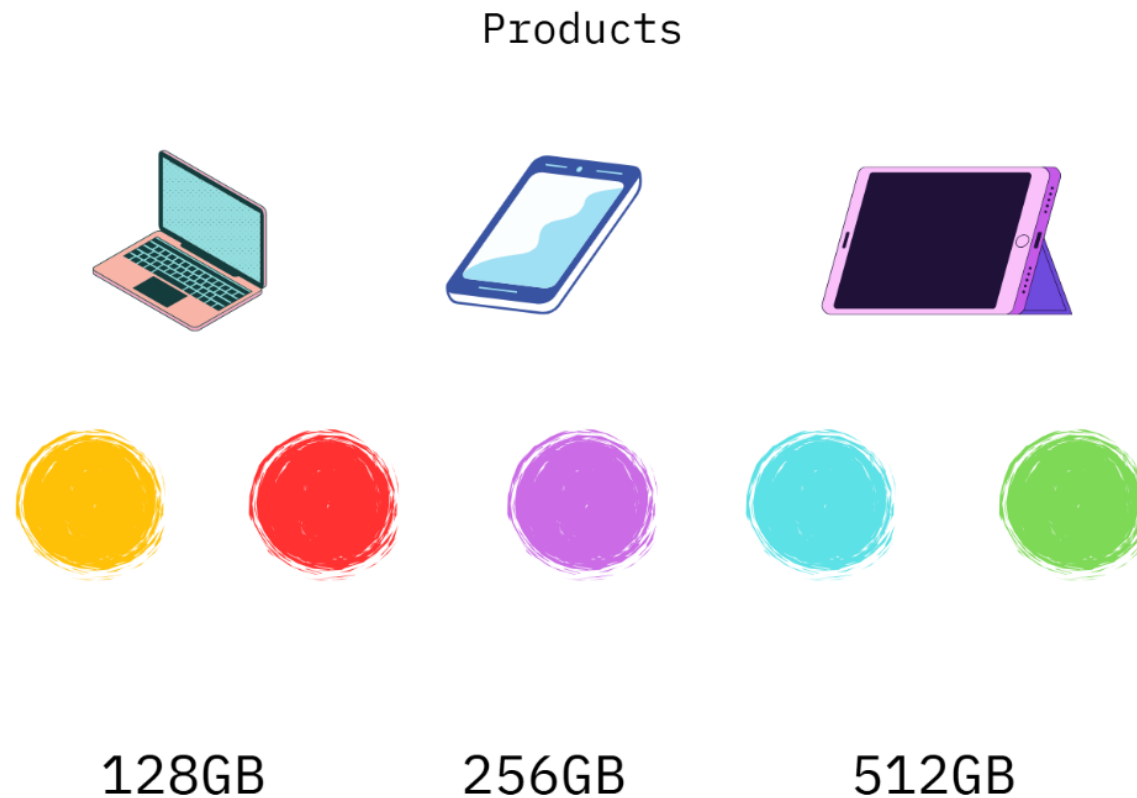
```
1 from itertools import cycle
2 COUNT = 4
3 my_iterator = cycle("ABC")
4 for _i in range(COUNT):
5     print(next(my_iterator))
```

A
B
C
A

✓ Program exited with code 0

Revision: Itertools (Product)

I have these options for devices, colors, and storage size. How many apple products can I come up with?



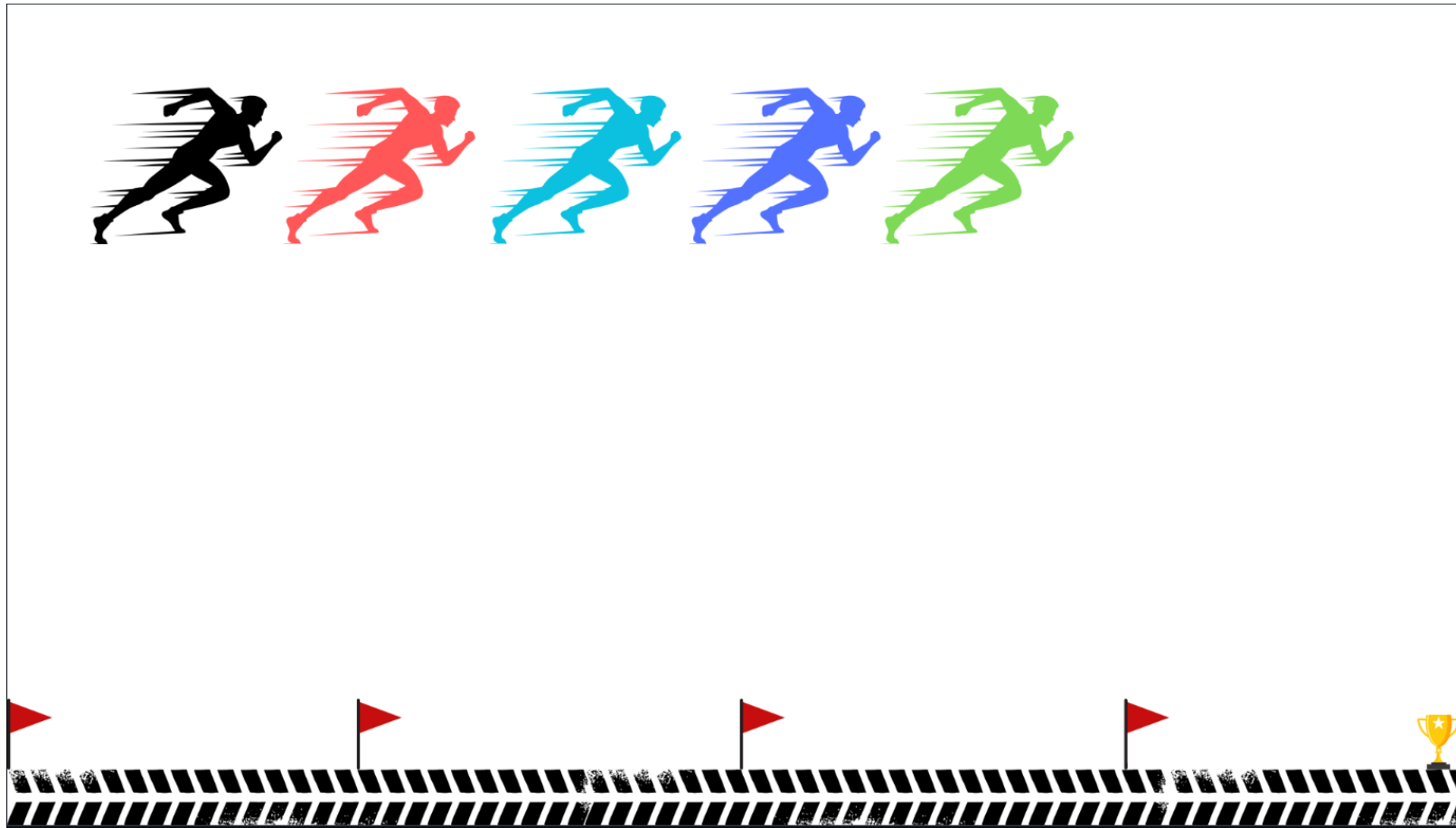
Revision: Itertools (Product)

```
1 import itertools
2
3 all_products = itertools.product(
4     ['laptop', 'phone', 'tablet'],
5     ['Y', 'R', 'P', 'B', 'G'],
6     [128, 256, 512]
7 )
8
9 print(list(all_products))
```

```
[('laptop', 'Y', 128), ('laptop', 'Y', 256), ('laptop', 'Y', 512), ('laptop', 'R', 128), ('laptop', 'R', 256), ('laptop', 'R', 512), ('laptop', 'P', 128), ('laptop', 'P', 256), ('laptop', 'P', 512), ('laptop', 'B', 128), ('laptop', 'B', 256), ('laptop', 'B', 512), ('laptop', 'G', 128), ('laptop', 'G', 256), ('laptop', 'G', 512), ('phone', 'Y', 128), ('phone', 'Y', 256), ('phone', 'Y', 512), ('phone', 'R', 128), ('phone', 'R', 256), ('phone', 'R', 512), ('phone', 'P', 128), ('phone', 'P', 256), ('phone', 'P', 512), ('phone', 'B', 128), ('phone', 'B', 256), ('phone', 'B', 512), ('phone', 'G', 128), ('phone', 'G', 256), ('phone', 'G', 512), ('tablet', 'Y', 128), ('tablet', 'Y', 256), ('tablet', 'Y', 512), ('tablet', 'R', 128), ('tablet', 'R', 256), ('tablet', 'R', 512), ('tablet', 'P', 128), ('tablet', 'P', 256), ('tablet', 'P', 512), ('tablet', 'B', 128), ('tablet', 'B', 256), ('tablet', 'B', 512), ('tablet', 'G', 128), ('tablet', 'G', 256), ('tablet', 'G', 512)]
```

Revision: Itertools (Permutations)

I want to know all possible orderings for a 4-man relay sprint from 5 candidates



Credit: Daksh Agrawal

Revision: Itertools (Permutations)

```
1 import itertools
2
3 p_teams = itertools.permutations(
4     iterable: ["A", "B", "C", "D", "E"],
5     r: 4
6 )
7
8 print(list(p_teams))
```

```
[('A', 'B', 'C', 'D'), ('A', 'B', 'C', 'E'), ('A', 'B', 'D', 'C'), ('A', 'B', 'D', 'E'), ('A', 'B', 'E', 'C'), ('A', 'B', 'E', 'D'), ('A', 'C', 'B', 'D'), ('A', 'C', 'B', 'E'), ('A', 'C', 'D', 'B'), ('A', 'C', 'D', 'E'), ('A', 'C', 'E', 'B'), ('A', 'C', 'E', 'D'), ('A', 'D', 'B', 'C'), ('A', 'D', 'B', 'E'), ('A', 'D', 'C', 'B'), ('A', 'D', 'C', 'E'), ('A', 'D', 'E', 'B'), ('A', 'D', 'E', 'C'), ('A', 'E', 'B', 'C'), ('A', 'E', 'B', 'D'), ('A', 'E', 'C', 'B'), ('A', 'E', 'C', 'D'), ('A', 'E', 'D', 'B'), ('A', 'E', 'D', 'C'), ('B', 'A', 'C', 'D'), ('B', 'A', 'C', 'E'), ('B', 'A', 'D', 'C'), ('B', 'A', 'D', 'E'), ('B', 'A', 'E', 'C'), ('B', 'A', 'E', 'D'), ('B', 'C', 'A', 'D'), ('B', 'C', 'A', 'E'), ('B', 'C', 'D', 'A'), ('B', 'C', 'D', 'E'), ('B', 'C', 'E', 'A'), ('B', 'C', 'E', 'D'), ('B', 'D', 'A', 'C'), ('B', 'D', 'A', 'E'), ('B', 'D', 'C', 'A'), ('B', 'D', 'C', 'E'), ('B', 'D', 'E', 'A'), ('B', 'D', 'E', 'C'), ('B', 'E', 'A', 'C'), ('B', 'E', 'A', 'D'), ('B', 'E', 'C', 'A'), ('B', 'E', 'C', 'D'), ('B', 'E', 'D', 'A'), ('B', 'E', 'D', 'C'), ('B', 'E', 'D', 'E'), ('C', 'A', 'B', 'D'), ('C', 'A', 'B', 'E'), ('C', 'A', 'D', 'B'), ('C', 'A', 'D', 'E'), ('C', 'A', 'E', 'B'), ('C', 'A', 'E', 'D'), ('C', 'B', 'A', 'D'), ('C', 'B', 'A', 'E'), ('C', 'B', 'D', 'A'), ('C', 'B', 'D', 'E'), ('C', 'B', 'E', 'A'), ('C', 'B', 'E', 'D'), ('C', 'C', 'D', 'A'), ('C', 'C', 'D', 'E'), ('C', 'C', 'E', 'A'), ('C', 'C', 'E', 'D'), ('C', 'D', 'A', 'B'), ('C', 'D', 'A', 'E'), ('C', 'D', 'B', 'A'), ('C', 'D', 'B', 'E'), ('C', 'D', 'E', 'A'), ('C', 'D', 'E', 'B'), ('C', 'E', 'A', 'D'), ('C', 'E', 'A', 'B'), ('C', 'E', 'B', 'A'), ('C', 'E', 'B', 'D'), ('C', 'E', 'D', 'A'), ('C', 'E', 'D', 'B'), ('C', 'E', 'D', 'E'), ('D', 'A', 'B', 'C'), ('D', 'A', 'B', 'E'), ('D', 'A', 'C', 'B'), ('D', 'A', 'C', 'E'), ('D', 'A', 'E', 'B'), ('D', 'A', 'E', 'C'), ('D', 'B', 'A', 'C'), ('D', 'B', 'A', 'E'), ('D', 'B', 'C', 'A'), ('D', 'B', 'C', 'E'), ('D', 'B', 'E', 'A'), ('D', 'B', 'E', 'C'), ('D', 'C', 'A', 'B'), ('D', 'C', 'A', 'E'), ('D', 'C', 'B', 'A'), ('D', 'C', 'B', 'E'), ('D', 'C', 'E', 'A'), ('D', 'C', 'E', 'B'), ('D', 'D', 'A', 'B'), ('D', 'D', 'A', 'E'), ('D', 'D', 'B', 'A'), ('D', 'D', 'B', 'E'), ('D', 'D', 'C', 'A'), ('D', 'D', 'C', 'E'), ('D', 'D', 'E', 'A'), ('D', 'D', 'E', 'B'), ('E', 'A', 'B', 'C'), ('E', 'A', 'B', 'D'), ('E', 'A', 'C', 'B'), ('E', 'A', 'C', 'D'), ('E', 'A', 'D', 'B'), ('E', 'A', 'D', 'C'), ('E', 'B', 'A', 'C'), ('E', 'B', 'A', 'D'), ('E', 'B', 'C', 'A'), ('E', 'B', 'C', 'D'), ('E', 'B', 'D', 'A'), ('E', 'B', 'D', 'C'), ('E', 'C', 'A', 'B'), ('E', 'C', 'A', 'D'), ('E', 'C', 'B', 'A'), ('E', 'C', 'B', 'D'), ('E', 'C', 'D', 'A'), ('E', 'C', 'D', 'B'), ('E', 'D', 'A', 'B'), ('E', 'D', 'A', 'C'), ('E', 'D', 'B', 'A'), ('E', 'D', 'B', 'C'), ('E', 'D', 'C', 'A'), ('E', 'D', 'C', 'B')]
```

Credit: Daksh Agrawal

Revision: Itertools (Combinations)

How many teams of 5 can I form with 7 basketball players?



Revision: Itertools (Combinations)

```
1 import itertools
2
3 p_teams = itertools.combinations(
4     iterable: ["A", "B", "C", "D", "E", "F", "G"],
5     r: 5
6 )
7
8 print(list(p_teams))
```

```
[('A', 'B', 'C', 'D', 'E'), ('A', 'B', 'C', 'D', 'F'), ('A', 'B', 'C', 'D', 'G'), ('A', 'B', 'C', 'E', 'F'), ('A', 'B', 'C', 'E', 'G'), ('A', 'B', 'C', 'F', 'G'), ('A', 'B', 'D', 'E', 'F'), ('A', 'B', 'D', 'E', 'G'), ('A', 'B', 'D', 'F', 'G'), ('A', 'B', 'E', 'F', 'G'), ('A', 'C', 'D', 'E', 'F'), ('A', 'C', 'D', 'E', 'G'), ('A', 'C', 'D', 'F', 'G'), ('A', 'C', 'E', 'F', 'G'), ('A', 'D', 'E', 'F', 'G'), ('B', 'C', 'D', 'E', 'F'), ('B', 'C', 'D', 'E', 'G'), ('B', 'C', 'D', 'F', 'G'), ('B', 'C', 'E', 'F', 'G'), ('B', 'D', 'E', 'F', 'G'), ('C', 'D', 'E', 'F', 'G')]
```

Revision: Groupby

▶ Run

PYTHON



```
1 from itertools import groupby
2
3 def get_first_letter(x):
4     return x[0]
5
6 my_iterable = groupby(("AB", "AD", "BA", "BC", "BD", "DD"), get_first_letter)
7 for category, contents in my_iterable:
8     # contents is an iterable, so needs to be converted into a list
9     print(category, list(contents))
```

```
A ['AB', 'AD']
B ['BA', 'BC', 'BD']
D ['DD']
```

✓ Program exited with code 0

TuteSheet W9 – Exercises Q1(a)

1. A list comprehension is a shortcut notation used to accomplish simple iteration tasks that construct a list in one line of code. List comprehensions are formed by wrapping a pair of brackets around `<expression> <for iteration statement> <optional if filter condition>`. The iteration statement will be run and for each iteration, the result of the expression will be added to the list. If a filter condition is included, the object will only be added if that condition evaluates to `True`.

Evaluate the following list comprehensions. For each one, also write some python code to generate the same list without using a comprehension.

(a) `[(name, 0) for name in ("evelyn", "alex", "sam")]`

Output: `[('evelyn', 0), ('alex', 0), ('sam', 0)]`

w/o using a list comprehension:

```
my_list = []
for name in ("evelyn", "alex", "sam"):
    my_list.append((name, 0))
```


TuteSheet W9 – Exercises Q1(b), (c)

Evaluate the following list comprehensions. For each one, also write some python code to generate the same list without using a comprehension.

(b) `[i**2 for i in range(5) if i % 2 == 1]`

Output: `[1, 9]`

w/o using a list comprehension:

```
my_list = []
for i in range(5):
    if i % 2 == 1:
        my_list.append(i**2)
```

(c) `"".join([letter.upper() for letter in "python"])`

Output: `'PYTHON'`

w/o using a list comprehension:

```
my_list = []
for letter in "python":
    my_list.append(letter.upper())
my_str = "".join(my_list)
```

if we don't require the list: `my_str = "python".upper()`



TuteSheet W9 – Exercises Q1(d)

Evaluate the following list comprehensions. For each one, also write some python code to generate the same list without using a comprehension.

(d) `[(row, col) for row in range(3, 5) for col in range(2)]`

Output: `[(3, 0), (3, 1), (4, 0), (4, 1)]`

w/o using a list comprehension:

```
my_list = []
for row in range(3, 5):
    for col in range(2):
        my_list.append((row, col))
```



TuteSheet W9 – Exercises Q2

2. What happens if we use curly brackets instead of square brackets around a “list” comprehension? What happens if we use parentheses?

If we use curly brackets `{ }` around a comprehension syntax, it will evaluate into a **set**.

(e.g.) `{n**2 for n in range(1, 10)}`

`{1, 4, 9, 16, 25, 36, 49, 64, 81}`

If we use the key:value syntax for the expression part of a comprehension with curly brackets `{ }`, it will evaluate into a **dictionary**.

(e.g.) `{word:word.upper() for word in ["apple", "banana"]}`

`{'apple': 'APPLE', 'banana': 'BANANA'}`

If we use parentheses `( )`, it creates an object called a **generator, NOT** a **tuple**.

A **generator object** is an iterator that produces values one at a time, only when you ask for them, usually with a loop or by calling `next()`.

TuteSheet W9 – Exercises Q3

3. For a list such as `words = ['pencil', 'highlighter', 'paper-clip', 'ruler', 'pen']`, write a comprehension that gives the following:

(a) A list containing only the words that start with 'p'

```
[word for word in words if word.startswith('p')]
```

output: `['pencil', 'paper-clip', 'pen']`

(b) A dictionary mapping each word to their length

```
{word: len(word) for word in words}
```

output: `{ 'pencil':6, 'highlighter':11, 'paper-clip':10, 'ruler':5, 'pen':3 }`

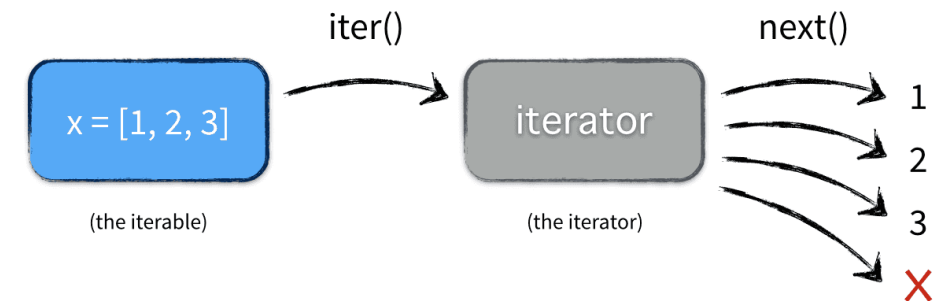
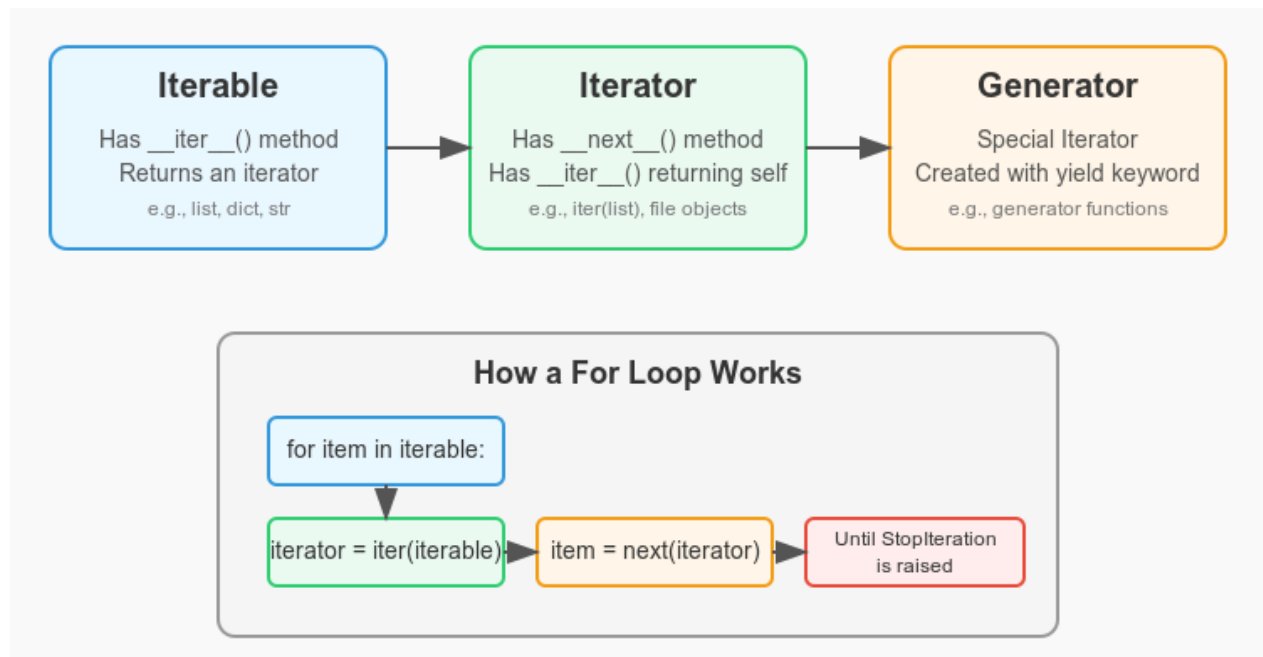
(c) A set of every character used in words

```
{letter for word in words for letter in word}
```

output: `{ '-', 'a', 'c', 'e', 'g', 'h', 'i', 'l', 'n', 'p', 'r', 't', 'u' }`

TuteSheet W9 – Exercises Q4(a)

- 4.
- An *iterable* is an object capable of returning its members one at a time. Examples of iterables include all sequence types (such as `list`, `str`, and `tuple`) and some non-sequence types like `dict` and `set`, and file objects.
 - An *iterator* is an object representing a stream of data. Repeated calls of `next(iterator)` return successive items in the stream. When no more items are available a `StopIteration` exception is raised. We can convert an iterable into an iterator using the `iter()` function.



TuteSheet W9 – Exercises Q4(a)

- 4.
- An *iterable* is an object capable of returning its members one at a time. Examples of iterables include all sequence types (such as `list`, `str`, and `tuple`) and some non-sequence types like `dict` and `set`, and file objects.
 - An *iterator* is an object representing a stream of data. Repeated calls of `next(iterator)` return successive items in the stream. When no more items are available a `StopIteration` exception is raised. We can convert an iterable into an iterator using the `iter()` function.

Convert these iterable objects into iterators and extract two elements into `first` and `second` variables:

(a) `iterable = "ABCDEFGH"`

A:

```
iterable = "ABCDEFGH"
iterator = iter(iterable)
first = next(iterator) # 'A'
second = next(iterator) # 'B'
```

- *iterable* : a `str`.
- `iter()` on a **string**: returns **one character at a time**, starting from index 0.
- `next()` : the next character from the iterator

TuteSheet W9 – Exercises Q4(b)

- 4.
- An *iterable* is an object capable of returning its members one at a time. Examples of iterables include all sequence types (such as `list`, `str`, and `tuple`) and some non-sequence types like `dict` and `set`, and file objects.
 - An *iterator* is an object representing a stream of data. Repeated calls of `next(iterator)` return successive items in the stream. When no more items are available a `StopIteration` exception is raised. We can convert an iterable into an iterator using the `iter()` function.

Convert these iterable objects into iterators and extract two elements into `first` and `second` variables:

(b) `iterable = {(0, 0), (0, 1), (1, 0), (1, 1)}`

A:

```
iterable = {(0, 0), (0, 1), (1, 0), (1, 1)}  
iterator = iter(iterable)  
first = next(iterator) #e.g., (0, 1)  
second = next(iterator) #e.g., (1, 0)
```

- ***iterable*** : a `set` of tuples.
- Since sets are **unordered**, the iteration order is **not guaranteed**.
- `iter()` on a **set**: returns **the elements one by one**.
- `next()` : the next tuple from the iterator



TuteSheet W9 – Exercises Q5

5. The `itertools` library provides functions that create memory-efficient iterators for a variety of sequences and combinatoric sets. Have a look at the Python documentation page for the `itertools` library and the `itertools` session in Ed Worksheet 14. The ones we will focus on are `cycle`, `product`, `permutations`, `combinations`, and `groupby`.

- Q6** `cycle()` returns an iterator that **repeats its elements indefinitely**.
(e.g. `cycle('ABC')` → `A B C A B C A B C ...`)
- Q7** `product()` returns an iterator that generates **all possible ordered pairs, as tuples (similar to nested for loops)** (e.g. `product('ABC', 'xy')` → `Ax Ay Bx By Cx Cy`)
- Q8** `permutations()` returns an iterator of **all possible ordered permutations of length r**.
(e.g. `permutations('ABC', 2)` → `AB AC BA BC CA CB`)
- Q8** `combinations()` returns an iterator of **all possible unordered combinations of length r, without repetition**.
(e.g. `combinations('ABC', 2)` → `AB AC BC`)
- Q9** `groupby()` returns an iterator that generates **consecutive keys and groups** from the iterable.
(e.g. `groupby(['A','B','DEF'], len)` → `(1, A B) (3, DEF)`)

TuteSheet W9 – Exercises Q6

6. What output does the following code print? Try changing the `while` loop to get the same output.

```
import itertools
beatboxer = itertools.cycle(['boots', 'and', 'cats', 'and'])

for count in range(9):
    print(next(beatboxer))
```

output: will print two iterations
of boots and cats and,
and will end with boots

boots
and
cats
and
boots
and
cats
and
boots

Using `while` loop

```
import itertools
beatboxer = itertools.cycle(['boots', 'and', 'cats', 'and'])

COUNT = 9
while COUNT:
    print(next(beatboxer))
    COUNT -= 1
```

TuteSheet W9 – Exercises Q7

7. A comedy series has episode names in an <animal> in <place> format, for example, “Elephants in Melbourne”. Using a single loop, write some code to print out every possible episode name, given:

```
animals = ['cats', 'dogs', 'hamsters', 'elephants']  
places = ['Melbourne', 'space', 'the supermarket']
```

```
import itertools  
  
animals = ['cats', 'dogs', 'hamsters', 'elephants']  
places = ['Melbourne', 'space', 'the supermarket']  
  
for animal, place in itertools.product(animals, places):  
    print(f"{animal.title()} in {place}")
```

```
Cats in Melbourne  
Cats in space  
Cats in the supermarket  
Dogs in Melbourne  
Dogs in space  
Dogs in the supermarket  
Hamsters in Melbourne  
Hamsters in space  
Hamsters in the supermarket  
Elephants in Melbourne  
Elephants in space  
Elephants in the supermarket
```



TuteSheet W9 – Exercises Q8

8. Compare the output of this code. What do you notice about the difference between combinations and permutations?

```
import itertools

numbers = [1, 2, 3]
print("combinations:", list(itertools.combinations(numbers, 2)))
print("permutations:", list(itertools.permutations(numbers, 2)))
```

```
combinations: [(1, 2), (1, 3), (2, 3)]
permutations: [(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```

combinations() returns an iterator of **all possible unordered combinations of length r, without repetition.**

*In this example, combinations show the ways of **choosing 2 elements** from the numbers list, but **the order** they are selected **isn't taken in to account**. Choosing 1 then 2 is the **same** as choosing 2 then 1, so only the first is included as a tuple (1, 2).*

permutations() returns an iterator of **all possible ordered permutations of length r.**

*In this example, Permutations show the different ways of choosing the 2 elements as a sequence, so the **order** that they are chosen **matters**. Choosing 1 then 2 is **different** to choosing 2 then 1, so both (1, 2) and (2, 1) are included in the result.*

TuteSheet W9 – Exercises Q9

9. What output does the following code print? What happens if we don't sort the `aussie_animals` list before doing `groupby`?

```
import itertools

aussie_animals = ["Possum", "Echidna", "Emu", "Koala", "Platypus", "Wombat"]

for key, group in itertools.groupby(sorted(aussie_animals), lambda x: x[0]):
    print(key, list(group))
```

iterable *key*

If we don't sort the `aussie_animals` list before doing `groupby`, then the output is:

E ['Echidna', 'Emu']
K ['Koala']
P ['Platypus', 'Possum']
W ['Wombat']

P ['Possum']
E ['Echidna', 'Emu']
K ['Koala']
P ['Platypus']
W ['Wombat']

It generates a break or new group every time the value of the key function changes (which is why it is usually necessary to have sorted the data using the same key function)

Independent Work

- **Next due dates:**

- Your **Project 2** will be released **this week**.
 - For any questions, please go to the **CIS Student Hub every weekday** in Level 3, Melbourne Connect or ask in the **Ed Discussion** Forums!
 - We can only provide **very limited**, general guidance.
- Ed Worksheets **14** and **15** is **due next Monday, May 11th, 6pm**.

- **Raise your hand** if you have any questions!

Scan here for annotated slides

